

Listing des programmes Python

8 juin 2026

Table des matières

1	../src/utils.py	2
2	../src/models.py	4
3	../src/kovasznyay.py	5
4	../src/kovasznyaydeeponet.py	6
5	../src/data.py	8
6	../src/annaive.py	9
7	../src/andata.py	12
8	../src/anspeedtransfer.py	14

1 ../src/utils.py

```
import json

import h5py
import torch
import pyvista as pv
import numpy as np
from matplotlib import pyplot as plt, tri
import matplotlib.colors as mcolors

def kova_solution_np(x, y, nu):
    l = 1 / (2 * nu) - np.sqrt(1 / (4 * nu**2) + 4 * np.pi**2)

    u = 1 - np.exp(1 * x) * np.cos(2 * np.pi * y)
    v = (1 / (2 * np.pi)) * np.exp(1 * x) * np.sin(2 * np.pi * y)
    p = 0.5 * (1 - np.exp(2 * l * x))

    return np.stack([u, v, p], axis=1)

def kova_solution(x, y, nu) -> torch.Tensor:
    result_np = kova_solution_np(x.detach().cpu().numpy(), y.detach().cpu().numpy(), nu)

    return torch.tensor(result_np, dtype=x.dtype, device=x.device)

def load_losses(filename):
    losses = []

    with open(filename, "r") as f:
        losses = [i for i in json.load(f) if np.shape(i)]

    return np.array(losses)

def ns_residuals(coords, u, v, p, nu):
    grad_u = torch.autograd.grad(u, coords, torch.ones_like(u), create_graph=True)[0]
    du_x = grad_u[:, 0]
    du_y = grad_u[:, 1]

    grad_v = torch.autograd.grad(v, coords, torch.ones_like(v), create_graph=True)[0]
    dv_x = grad_v[:, 0]
    dv_y = grad_v[:, 1]

    grad_p = torch.autograd.grad(p, coords, torch.ones_like(p), create_graph=True)[0]
    dp_x = grad_p[:, 0]
    dp_y = grad_p[:, 1]

    du_xx = torch.autograd.grad(du_x, coords, torch.ones_like(du_x), create_graph=True)[0][:, 0]
    du_yy = torch.autograd.grad(du_y, coords, torch.ones_like(du_y), create_graph=True)[0][:, 1]

    dv_xx = torch.autograd.grad(dv_x, coords, torch.ones_like(dv_x), create_graph=True)[0][:, 0]
    dv_yy = torch.autograd.grad(dv_y, coords, torch.ones_like(dv_y), create_graph=True)[0][:, 1]

    ns_x = u * du_x + v * du_y + dp_x - nu * (du_xx + du_yy)
    ns_y = u * dv_x + v * dv_y + dp_y - nu * (dv_xx + dv_yy)
    continuity = du_x + dv_y

    return ns_x, ns_y, continuity

def extract_ds_values(file_name, T):
    with h5py.File(file_name, "r") as f:
        coords = f["data0"][:]
        connectivity = f["data1"][:]
```

```

vitesse = f[f"data{2 + 2 * T}"][:]

cells = np.hstack([np.full((len(connectivity), 1), 4), connectivity]).ravel()
cell_types = np.full(len(connectivity), pv.CellType.TETRA)
mesh = pv.UnstructuredGrid(cells, cell_types, coords)
mesh.point_data["Vitesse"] = vitesse

slice_plane = mesh.slice(normal='z')

points = slice_plane.points[:, :2]
U = slice_plane.point_data["Vitesse"]
u = U[:, 0]
v = U[:, 1]

uv_list = np.stack([u, v], axis=1)
x = points[:, 0]
y = points[:, 1]

x = np.array(list(map(lambda x_c: x_c + 10, x)))
return x, y, uv_list

def extract_ds_outside_values(file_name):
    with h5py.File(file_name, "r") as f:
        coords = f["data0"][:]
        connectivity = f["data1"][:]

    cells = np.hstack([np.full((len(connectivity), 1), 4), connectivity]).ravel()
    cell_types = np.full(len(connectivity), pv.CellType.TETRA)
    mesh = pv.UnstructuredGrid(cells, cell_types, coords)

    slice_plane = mesh.slice(normal='z')
    edges = slice_plane.extract_feature_edges(boundary_edges=True,
                                              non_manifold_edges=False,
                                              feature_edges=False,
                                              manifold_edges=False)

    contour_points = edges.points
    x = np.array(list(map(lambda x_c: x_c + 10, contour_points[:, 0])))
    return x, contour_points[:, 1]

def plot_field(x, y, z, plot_title, z_label, show_norm=True):
    triang = tri.Triangulation(x, y)
    triangles = triang.triangles

    x_tri = x[triangles]
    y_tri = y[triangles]

    a = np.sqrt((x_tri[:, 1] - x_tri[:, 0]) ** 2 + (y_tri[:, 1] - y_tri[:, 0]) ** 2)
    b = np.sqrt((x_tri[:, 2] - x_tri[:, 1]) ** 2 + (y_tri[:, 2] - y_tri[:, 1]) ** 2)
    c = np.sqrt((x_tri[:, 0] - x_tri[:, 2]) ** 2 + (y_tri[:, 0] - y_tri[:, 2]) ** 2)

    max_edge = np.max(np.stack([a, b, c], axis=1), axis=1)

    threshold = 0.05
    triang.set_mask(max_edge > threshold)

    # --- Plot ---
    fig, ax = plt.subplots()
    if show_norm:
        norm = mcolors.TwoSlopeNorm(vmin=z.min(), vcenter=0, vmax=z.max())
        tcf = ax.tripcolor(triang, z, cmap='RdBu_r', shading='gouraud', norm=norm)
    else:
        tcf = ax.tripcolor(triang, z, cmap='viridis', shading='gouraud')
    ax.set_aspect('auto')
    ax.set_xlim(x.min() - 0.05, x.max() + 0.05)
    ax.set_ylim(y.min() - 0.05, y.max() + 0.05)

```

```

plt.colorbar(tcf, ax=ax, label=z_label)
plt.title(plot_title)
plt.tight_layout()
plt.xlabel("x")
plt.ylabel("y")
plt.show()

def kova_data():
    outside_points = torch.Tensor(
        [[-0.5, i] for i in np.linspace(-0.5, 1.5, 25)] +
        [[1, i] for i in np.linspace(-0.5, 1.5, 25)] +
        [[i, -0.5] for i in np.linspace(-0.5, 1, 25)] +
        [[i, 1.5] for i in np.linspace(-0.5, 1, 25)]).double()
    inside_points = torch.tensor(
        np.column_stack((np.random.uniform(-0.5, 1.0, 2000), np.random.uniform(-0.5, 1.5, 2000)))
    ).double().requires_grad_(True)

    return outside_points, inside_points

def u_func(x, y):
    return 1 - torch.exp(1 * x) * torch.cos(2 * torch.pi * y)

def v_func(x, y):
    return (1 / (2 * torch.pi)) * torch.exp(1 * x) * torch.sin(2 * np.pi * y)

def p_func(x):
    return 1 / 2 * (1 - torch.exp(2 * 1 * x))

def parabolic_inlet(x, V_max):
    x_center = (x.min() + x.max()) / 2
    R = (x.max() - x.min()) / 2

    return V_max * (1 - ((x - x_center) / R) ** 2)

def increase_points_number(points, n_variations=8, step_size=0.05, noise_std=0.02):
    augmented = [points]

    for _ in range(n_variations):
        # Pas aléatoire dans [-step_size, step_size]
        random_step = (torch.rand_like(points) * 2 - 1) * step_size
        # Ajouter un peu de bruit
        noise = torch.randn_like(points) * noise_std
        # Nouveaux points
        new_points = points + random_step + noise
        augmented.append(new_points)

    return torch.cat(augmented, dim=0)

```

2 ../src/models.py

```

import torch.nn as nn
import torch

class KovaDeepONet(nn.Module):
    def __init__(self, p=128, re_min=20, re_max=100):
        super(KovaDeepONet, self).__init__()
        self.p = p
        self.re_min = re_min
        self.re_max = re_max

        self.trunk_net = MLP(4, 128, 2, p, final_tanh=False)
        self.branch_net = MLP(4, 128, 1, p * 3)

        self.bias = nn.Parameter(torch.zeros(3).double())
        self._init_weights()

```

```

def _init_weights(self):
    for m in self.modules():
        if isinstance(m, nn.Linear):
            nn.init.xavier_normal_(m.weight)
            nn.init.zeros_(m.bias)

def forward(self, re, points):
    re_norm = 2 * (re - self.re_min) / (self.re_max - self.re_min) - 1

    b = self.branch_net(re_norm.unsqueeze(-1)).reshape(-1, 3, self.p)
    t = self.trunk_net(points)

    out = torch.einsum('bcp,np->bcn', b, t)
    return out.permute(0, 2, 1) + self.bias

class MLP(nn.Module):
    def __init__(self, hidden_n, layer_depth, input_n, output_n, final_tanh=False):
        super(MLP, self).__init__()
        self.layers = nn.ModuleList()
        self.final_tanh = final_tanh

        self.layers.append(nn.Linear(input_n, layer_depth))

        for i in range(hidden_n):
            self.layers.append(nn.Linear(layer_depth, layer_depth))

        self.layers.append(nn.Linear(layer_depth, output_n))

    def forward(self, x):
        for i in range(len(self.layers) - 1):
            x = torch.tanh(self.layers[i](x))

        x = self.layers[-1](x)
        if self.final_tanh:
            x = torch.tanh(x) # Tanh final optionnel
        return x

```

3 ../src/kovaszney.py

```

import torch
import torch.nn as nn
import numpy as np

import json

from src.utils import ns_residuals, kova_data, u_func, v_func, p_func

Re = 40
nu = 1 / Re
l = 1 / (2 * nu) - np.sqrt(1 / (4 * nu ** 2) + 4 * np.pi ** 2)

PINNModel = nn.Sequential(
    nn.Linear(2, 64),
    nn.Tanh(),
    nn.Linear(64, 64),
    nn.Tanh(),
    nn.Linear(64, 64),
    nn.Tanh(),
    nn.Linear(64, 3)
).double()

def compute_total_loss():
    inside_values = PINNModel(inside_points)
    ns_x, ns_y, continuity = ns_residuals(inside_points, inside_values[:, 0], inside_values[:, 1],
    ↪ inside_values[:, 2], 1/Re)

```

```

ns_loss_x_inside, ns_loss_y_inside = mse(ns_x, torch.zeros_like(ns_x)), mse(ns_y,
↪ torch.zeros_like(ns_y))
continuity_loss_inside = mse(continuity, torch.zeros_like(continuity))

outside_values = PINNModel(outside_points)
outside_loss = mse(outside_values, outside_true_results)

losses.append([ns_loss_x_inside.item(), ns_loss_y_inside.item(), continuity_loss_inside.item(),
↪ outside_loss.item(), ns_loss_x_inside.item() + ns_loss_y_inside.item() +
↪ continuity_loss_inside.item() + outside_loss.item()])

return ns_loss_x_inside + ns_loss_y_inside + continuity_loss_inside + 100 * outside_loss

adam_optimizer = torch.optim.Adam(PINNModel.parameters(), lr=0.001)
optimizer_lbfgs = torch.optim.LBFGS(
    PINNModel.parameters(),
    lr=1.0,
    max_iter=20,
    history_size=50,
    tolerance_grad=1e-7,
    tolerance_change=1e-9,
    line_search_fn="strong_wolfe"
)

mse = nn.MSELoss()

outside_points, inside_points = kova_data()

outside_true_results = torch.stack([
    u_func(outside_points[:, 0], outside_points[:, 1]),
    v_func(outside_points[:, 0], outside_points[:, 1]),
    p_func(outside_points[:, 0])
], dim=1)

adam_epochs = 1000
lbfgs_epochs = 1000

losses = []

for _ in range(adam_epochs):
    adam_optimizer.zero_grad()
    loss = compute_total_loss()
    loss.backward()
    losses.append(loss.item())
    adam_optimizer.step()

for _ in range(lbfgs_epochs):
    def closure():
        optimizer_lbfgs.zero_grad()
        loss = compute_total_loss()
        loss.backward()
        return loss

    loss = optimizer_lbfgs.step(closure)

torch.save(PINNModel.state_dict(), "./PINN_Model_09.pt")

with open("losses_09.json", "w") as f:
    json.dump(losses, f)

```

4 ../src/kovasznydeeponet.py

```

import torch
import torch.nn as nn
import numpy as np

```

```

import json

from models import KovaDeepONet
from utils import kova_solution, ns_residuals

def loss_function(re_values):
    ns_total_loss = torch.tensor(0.0).double()
    bc_total_loss = torch.tensor(0.0).double()

    for re in re_values:
        re_t = torch.tensor([re]).double()
        nu = 1.0 / re

        inside_points = torch.tensor(
            np.column_stack((np.random.uniform(-0.5, 1.0, 2000), np.random.uniform(-0.5, 1.5, 2000)))
        ).double().requires_grad_(True)

        inside_results = model(re_t, inside_points).squeeze(0)

        ns_x, ns_y, continuity = ns_residuals(inside_points, inside_results[:, 0], inside_results[:, 1],
            ↪ inside_results[:, 2], nu)
        ns_total_loss += (mse(ns_x, torch.zeros_like(ns_x)) + mse(ns_y, torch.zeros_like(ns_y)) +
            ↪ mse(continuity, torch.zeros_like(continuity)))

        outside_results = model(re, outside_points).squeeze(0)
        labels = kova_solution(outside_points[:, 0], outside_points[:, 1], nu)
        bc_total_loss += mse(outside_results, labels)

    n = len(re_values)
    return LAMBDA_NS * ns_total_loss / n, LAMBDA_BC * bc_total_loss / n

mse = nn.MSELoss()

P          = 128    # Dimension de l'espace latent branch/trunk
N_COLLOC   = 2000   # Points intérieurs par Re
N_BC       = 25     # Points par côté frontière
RE_RANGE   = (20, 100)

LAMBDA_NS = 1.0
LAMBDA_BC = 50.0

RE_BATCH_SIZE = 8

outside_points = torch.Tensor([[[-0.5, i] for i in np.linspace(-0.5, 1.5, 25)] + [[1, i] for i in
    ↪ np.linspace(-0.5, 1.5, 25)] + [[i, -0.5] for i in np.linspace(-0.5, 1, 25)] + [[i, 1.5] for i in
    ↪ np.linspace(-0.5, 1, 25)]]).double()

model = KovaDeepONet().double()

losses = []

optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=2000)

for epoch in range(5000):
    # Échantillonnage aléatoire de Re à chaque époque
    re_batch = np.random.uniform(*RE_RANGE, RE_BATCH_SIZE).tolist()

    optimizer.zero_grad()
    loss_ns, loss_bc = loss_function(re_batch)
    loss = loss_ns + loss_bc
    losses.append([loss_ns.item(), loss_bc.item(), loss.item()])
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()

```

```

scheduler.step()

if epoch % 500 == 0:
    print(f"[epoch:5d] NS={loss_ns:.2e} | BC={loss_bc:.2e}")

torch.save(model.state_dict(), "./DON_kova.pt")

with open("losses_09.json", "w") as f:
    json.dump(losses, f)

```

5 ../src/data.py

```

import sys

import matplotlib.pyplot as plt
import numpy as np
import pyvista as pv

sys.setrecursionlimit(10000)

class AneurysmData:
    def __init__(self, file, w, h):
        mesh = pv.read(file)
        slice_plane = mesh.slice(normal='z')

        edges = slice_plane.extract_feature_edges(boundary_edges=True)
        contour_points = np.array(edges.points)
        contour_points = np.array([[cell[1], cell[0] + 10] for cell in contour_points])

        self.file = file
        self.w = w
        self.h = h
        self.s_h = 14 / self.h
        self.s_w = 20 / self.w

        self.grid = np.array([[0 for _ in range(self.w)] for _ in range(self.h)])

        for point in contour_points:
            nearest_cell_x = int(np.floor(point[1] / self.s_w))
            nearest_cell_y = int(np.floor(point[0] / self.s_h))
            self.grid[nearest_cell_y][nearest_cell_x] = 1

        self.__filling(int(h / 2), int(w / 2))
        self.grid[int(h / 2)][int(w / 2)] = 2
        self.__filling_outside()

    @property
    def cells(self):
        r = []
        for i, row in enumerate(self.grid):
            for j, cell in enumerate(row):
                r.append([j, i, cell])
        return r

    @property
    def outside(self):
        return [cell for cell in self.cells if cell[2] == 1]

    @property
    def d_outside(self):
        return np.array([[x * (0.02 / w), y * (0.015 / h), v] for x, y, v in self.cells if v == 1])

    @property
    def inside(self):
        return np.array([cell for cell in self.cells if cell[2] == 2])

```



```

@property
def d_inside(self):
    return np.array([[x * (0.02 / w), y * (0.015 / h), v] for x, y, v in self.cells if v == 2])

@property
def walls(self):
    return np.array([cell for cell in self.cells if cell[2] == 3])

@property
def d_walls(self):
    return np.array([[x * (0.02 / w), y * (0.015 / h), v] for x, y, v in self.cells if v == 3])

def is_outline(self, cell):
    return self.grid[cell[1]][cell[0]] == 1

def is_inside(self, cell):
    return self.grid[cell[1]][cell[0]] == 2

def is_true_outside(self, cell):
    return self.grid[cell[1]][cell[0]] == 3

def get_coords(self, i, j):
    return i * self.s_w, j * self.s_h

def __filling(self, i, j):
    neigs = [(i + 1, j), (i - 1, j), (i, j - 1), (i, j + 1)]
    for pixel in neigs:
        if 0 <= pixel[0] < self.h and 0 <= pixel[1] < self.w:
            if self.grid[pixel[0]][pixel[1]] == 0:
                self.grid[pixel[0]][pixel[1]] = 2
                self.__filling(pixel[0], pixel[1])

def __filling_outside(self):
    for (i, j, _) in self.outside:
        neigs = [(i + 1, j), (i - 1, j), (i, j - 1), (i, j + 1)]
        for pixel in neigs:
            if 0 <= pixel[1] < self.h and 0 <= pixel[0] < self.w and self.is_inside(pixel):
                self.grid[j][i] = 3
                break

def plot(self, show_outline=False):
    for cell in self.cells:
        if self.is_outline(cell) and show_outline:
            plt.gca().add_patch(
                plt.Rectangle(self.get_coords(cell[0], cell[1]), self.s_w, self.s_h, fill=True,
                    ↪ color='b'))
        if self.is_inside(cell):
            plt.gca().add_patch(
                plt.Rectangle(self.get_coords(cell[0], cell[1]), self.s_w, self.s_h, fill=True,
                    ↪ color='r'))
        elif self.is_true_outside(cell):
            plt.gca().add_patch(
                plt.Rectangle(self.get_coords(cell[0], cell[1]), self.s_w, self.s_h, fill=True,
                    ↪ color='g'))
        else:
            plt.gca().add_patch(
                plt.Rectangle(self.get_coords(cell[0], cell[1]), self.s_w, self.s_h, fill=False, lw=0))
    plt.title("Points du contour")
    plt.axis('equal')
    plt.show()

```

6 ../src/annaive.py

```
import sys
```

```

import numpy as np

import json

import torch
import torch.nn as nn
from torch.optim.lr_scheduler import StepLR

from src.data import AneurysmData
from src.utils import ns_residuals

sys.setrecursionlimit(10000)

w, h = 150, 90
data = AneurysmData("../data/Solid_77.vtk", w, h)

rho = 1055.0
mu = 0.0035
nu = mu / rho

V_max = 1.114

L_ref = 0.02
P_ref = rho * V_max ** 2 # Pa

Re = V_max * 0.004 / nu

nu = 1 / Re

inlet_radius = 0.09

def increase_points_number(points, n_variations=8, step_size=0.05, noise_std=0.02):
    augmented = [points]

    for _ in range(n_variations):
        # Pas aléatoire dans [-step_size, step_size]
        random_step = (torch.rand_like(points) * 2 - 1) * step_size
        # Ajouter un peu de bruit
        noise = torch.randn_like(points) * noise_std
        # Nouveaux points
        new_points = points + random_step + noise
        augmented.append(new_points)

    return torch.cat(augmented, dim=0)

def compute_inlet_velocity(points):
    dx = points[:, 0] - 0.19
    dy = points[:, 1]
    distance_sq = dx * dx + dy * dy

    velocity = (1 - distance_sq / (inlet_radius ** 2))

    return velocity

inside_points = list(np.float64(np.column_stack((data.d_inside[:, 0] / L_ref, data.d_inside[:, 1] /
↵ L_ref))))
inside_points = torch.tensor(inside_points).double()

inside_points = increase_points_number(inside_points, 2, 0.001, 0.001)
inside_points = inside_points.detach().requires_grad_(True)

inlet_points = filter_inlet_points(inside_points.detach())
inlet_points = increase_points_number(inlet_points, 8, 0.001, 0.001)
inlet_points = inlet_points.detach().requires_grad_(True)

```

```

outside_points = torch.tensor(np.float64(np.column_stack((data.d_walls[:, 0] / L_ref, data.d_walls[:, 1] /
↪ L_ref)))).double()
outlet_points = torch.tensor(np.float64(np.array([[x * (0.02 / w) / L_ref, y * (0.015 / h) / L_ref] for x,
↪ y, _ in
data.inside[(data.inside[:, 1] == 0) & (data.inside[:, 0]
↪ > 50)]])).double()

inlet_labels = torch.column_stack((
    torch.zeros(len(inlet_points)),
    compute_inlet_velocity(inlet_points.detach())
)).detach()

outside_points = increase_points_number(outside_points, 8, 0.001, 0.001).detach()
outlet_points = increase_points_number(outlet_points, 16, 0.001, 0.001).detach()

PINNModel = nn.Sequential(
    nn.Linear(2, 128), nn.Tanh(),
    nn.Linear(128, 128), nn.Tanh(),
    nn.Linear(128, 128), nn.Tanh(),
    nn.Linear(128, 128), nn.Tanh(),
    nn.Linear(128, 3)
).double()

def initialize_weights(module):
    if isinstance(module, nn.Linear):
        nn.init.xavier_uniform_(module.weight, gain=1.0)
        nn.init.zeros_(module.bias)

PINNModel.apply(initialize_weights)

mse = nn.MSELoss()
losses = []

def loss_function():
    inside_values = PINNModel(inside_points)
    ns_x, ns_y, continuity = ns_residuals(inside_points, inside_values[:, 0], inside_values[:, 1],
↪ inside_values[:, 2], nu)

    ns_loss = mse(ns_x, torch.zeros_like(ns_x)) + mse(ns_y, torch.zeros_like(ns_y))
    cont_loss = mse(continuity, torch.zeros_like(continuity))

    wall_values = PINNModel(outside_points)
    wall_loss = mse(wall_values[:, 0], torch.zeros_like(wall_values[:, 0])) \
        + mse(wall_values[:, 1], torch.zeros_like(wall_values[:, 1]))

    # Inlet
    inlet_values = PINNModel(inlet_points)
    inlet_loss = mse(inlet_values[:, 0], inlet_labels[:, 0]) \
        + mse(inlet_values[:, 1], inlet_labels[:, 1])

    outlet_values = PINNModel(outlet_points)
    outlet_loss = mse(outlet_values[:, 2], torch.zeros_like(outlet_values[:, 2]))

    total = (
        ns_loss
        + cont_loss
        + 50 * wall_loss
        + 50 * inlet_loss
        + 5 * outlet_loss
    )

    losses.append([ns_loss.item(), cont_loss.item(), wall_loss.item(),
        inlet_loss.item(), outlet_loss.item(), total.item()])
    return total

```

```

lossses = []

adam_epochs = 20000
lbfgs_epochs = 3000

adam_optimizer = torch.optim.Adam(PINNModel.parameters(), lr=0.001)
scheduler = StepLR(adam_optimizer, step_size=15000, gamma=0.1)

for epoch in range(adam_epochs):
    adam_optimizer.zero_grad()

    total_loss = loss_function()
    total_loss.backward()

    adam_optimizer.step()
    scheduler.step()

optimizer_lbfgs = torch.optim.LBFGS(
    PINNModel.parameters(),
    lr=1.0,
    max_iter=20,
    max_eval=25,
    tolerance_grad=1e-7,
    tolerance_change=1e-9,
    history_size=50,
    line_search_fn="strong_wolfe"
)

for _ in range(lbfgs_epochs):
    def closure():
        optimizer_lbfgs.zero_grad()
        total_loss = loss_function()
        total_loss.backward()

        return total_loss

    optimizer_lbfgs.step(closure)

torch.save(PINNModel.state_dict(), "./AN_model_01.pt")

with open("AN_model_01.json", "w") as f:
    json.dump(losses, f)

```

7 ../src/andata.py

```

import json

import numpy as np
import torch
import torch.nn as nn

from src.utils import extract_ds_values, ns_residuals

H5_FILE = "../coarse_dataset/AllFields_Resultats_MESH_210.h5"
T = 8 # pas de temps 20

RE = 520
nu = 1 / RE

Q = 7

P = -3700 + 1.32 * Q
P_ref = abs(P)
P_norm = P / P_ref

```

```

U_ref = 1000

model = nn.Sequential(
    nn.Linear(2, 256),
    nn.Tanh(),
    nn.Linear(256, 256),
    nn.Tanh(),
    nn.Linear(256, 256),
    nn.Tanh(),
    nn.Linear(256, 3)
).double()

optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

mse = nn.MSELoss()
losses = []
adam_epochs = 10000

x, y, uv = extract_ds_values(H5_FILE, T)

inside_points = torch.tensor(
    np.column_stack((x / 18, y / 14)),
).double().requires_grad_(True)

inside_labels = torch.tensor(uv / U_ref).double()
outlet_points = torch.column_stack((torch.linspace(14.5 / 18, 17.85 / 18, 50), torch.zeros(50))).double()

def compute_total_loss():
    inside_values = model(inside_points)
    inside_values_loss = mse(inside_values[:, :2], inside_labels)

    ns_x, ns_y, continuity = ns_residuals(inside_points, inside_values[:, 0], inside_values[:, 1],
    ↪ inside_values[:, 2], nu)

    ns_loss_x_inside, ns_loss_y_inside = mse(ns_x, torch.zeros_like(ns_x)), mse(ns_y,
    ↪ torch.zeros_like(ns_y))
    continuity_loss_inside = mse(continuity, torch.zeros_like(continuity))

    outlet_values = model(outlet_points)
    outlet_values_loss = mse(outlet_values[:, 2], torch.full_like(outlet_values[:, 2], P_norm).double())

    losses.append([
        inside_values_loss.item(),
        ns_loss_x_inside.item(),
        ns_loss_y_inside.item(),
        continuity_loss_inside.item(),
        outlet_values_loss.item(),
        ns_loss_x_inside.item() + ns_loss_y_inside.item() + continuity_loss_inside.item() +
        ↪ outlet_values_loss.item() + inside_values_loss.item()
    ])

    return ns_loss_x_inside + ns_loss_y_inside + continuity_loss_inside + 20 * outlet_values_loss + 100 *
    ↪ inside_values_loss

for _ in range(adam_epochs):
    optimizer.zero_grad()
    loss = compute_total_loss()
    loss.backward()
    optimizer.step()

optimizer_lbfgs = torch.optim.LBFGS(
    model.parameters(), lr=1.0, max_iter=50000,
    history_size=100, line_search_fn="strong_wolfe"
)

def closure():

```

```

optimizer_lbfgs.zero_grad()
loss = compute_total_loss()
loss.backward()
return loss

optimizer_lbfgs.step(closure)

torch.save(model.state_dict(), "./DS_Model_03.pt")

with open("DS_Model_03.json", "w") as f:
    json.dump(losses, f)

```

8 ../src/anspeedtransfer.py

```

import json

import numpy as np
import torch
import torch.nn as nn

from src.utils import extract_ds_values, increase_points_number, parabolic_inlet, extract_ds_outside_values,
↪ \
    ns_residuals

H5_FILE = "../input/datasets/eliepernet/mesh-210/AllFields_Resultats_MESH_210.h5"
MODEL_FILE = "../input/models/eliepernet/modelds/pytorch/default/1/DS_Model_03.pt"

T = 20

V_max = 0.777
P_fact = 0.521

Re = 377
nu = 1 / Re

PINNModel = nn.Sequential(
    nn.Linear(2, 256),
    nn.Tanh(),
    nn.Linear(256, 256),
    nn.Tanh(),
    nn.Linear(256, 256),
    nn.Tanh(),
    nn.Linear(256, 3)
).float()
PINNModel.load_state_dict(torch.load(MODEL_FILE))
frozen_params = {n: p.clone().detach() for n, p in PINNModel.named_parameters()}

optimizer_transfer = torch.optim.Adam(
    PINNModel.parameters(),
    lr=1e-5
)

mse = nn.MSELoss()
losses = []

x, y, _ = extract_ds_values(H5_FILE, T)

inside_points = torch.tensor(
    np.column_stack((x / 20, y / 14)),
).float()

inside_points = increase_points_number(inside_points, 4, 0.002, 0.002)
inside_points = inside_points.detach().requires_grad_(True)

inlet_points = torch.column_stack((
    torch.linspace(0.1, 0.285, 150),

```

```

        torch.zeros(150)
    ).float()
    inlet_points = inlet_points.detach().requires_grad_(True)

    inlet_labels = torch.column_stack((
        torch.zeros(len(inlet_points)),
        parabolic_inlet(inlet_points[:, 0].detach(), V_max)
    ).float().detach())

    x_out, y_out = extract_ds_outside_values(H5_FILE)
    outside_points = torch.tensor(
        np.column_stack((x_out / 20, y_out / 14))
    ).float()
    outside_points = outside_points[outside_points[:, 1] > 0.001].detach()

    outlet_points = torch.column_stack((
        torch.linspace(14.5 / 20, 17.85 / 20, 150),
        torch.zeros(150)
    ).float().detach())

def compute_total_loss(epoch, n_epochs, lambda_anchor=5e3):
    inside_values = PINNModel(inside_points)

    # BC
    inlet_values = PINNModel(inlet_points)
    outside_values = PINNModel(outside_points)
    outlet_values = PINNModel(outlet_points)

    inlet_loss = mse(inlet_values[:, :2], inlet_labels)
    outside_loss = mse(outside_values[:, :2], torch.zeros_like(outside_values[:, :2]))
    outlet_loss = mse(outlet_values[:, 2], torch.full_like(outlet_values[:, 2], P_fact))

    bc_loss = 50 * outside_loss + 50 * inlet_loss + 5 * outlet_loss

    ns_x, ns_y, continuity = ns_residuals(
        inside_points,
        inside_values[:, 0], inside_values[:, 1], inside_values[:, 2],
        nu
    )
    ns_loss = mse(ns_x, torch.zeros_like(ns_x)) \
        + mse(ns_y, torch.zeros_like(ns_y))
    cont_loss = mse(continuity, torch.zeros_like(continuity))

    w_pde = min(1.0, max(0.0, (epoch - 0.2 * n_epochs) / (0.4 * n_epochs)))
    pde_loss = w_pde * (ns_loss + cont_loss)

    pinn_loss = bc_loss + pde_loss

    anchor_loss = sum(
        (p - frozen_params[n]).pow(2).sum()
        for n, p in PINNModel.named_parameters()
    )

    total = pinn_loss + lambda_anchor * anchor_loss

    losses.append([
        total.item(),
        ns_loss.item(),
        cont_loss.item(),
        inlet_loss.item(),
        outside_loss.item(),
        outlet_loss.item(),
        anchor_loss.item(),
        w_pde,
        lambda_anchor
    ])

```

```

    return total, w_pde

def get_lambda(epoch, n_epochs, lambda_start=1e5, lambda_end=1e2):
    t = epoch / n_epochs
    return lambda_start * (lambda_end / lambda_start) ** t

transfer_epochs = 8000

for epoch in range(transfer_epochs):
    optimizer_transfer.zero_grad()
    lam = get_lambda(epoch, transfer_epochs)
    loss, w_pde = compute_total_loss(epoch, transfer_epochs, lambda_anchor=lam)
    loss.backward()
    optimizer_transfer.step()

last_losses = [l[0] for l in losses[-200:]]

variation = max(last_losses) - min(last_losses)
if variation < 0.05 * np.mean(last_losses):
    print("Adam stable → lancement L-BFGS")

    for p in PINNModel.parameters():
        p.requires_grad = True

    optimizer_lbfgs = torch.optim.LBFGS(
        PINNModel.parameters(), lr=1.0,
        max_iter=500, history_size=50,
        line_search_fn="strong_wolfe"
    )

    def closure_transfer():
        optimizer_lbfgs.zero_grad()
        loss, _ = compute_total_loss(transfer_epochs, transfer_epochs, lambda_anchor=0.0)
        loss.backward()
        return loss

    for step in range(20):
        optimizer_lbfgs.step(closure_transfer)

torch.save(PINNModel.state_dict(), "./DS_Model_03.pt")

with open("DS_Model_03.json", "w") as f:
    json.dump(losses, f)

```